

2 Moderne JavaScript-Grundlagen

Bevor wir uns mit TypeScript beschäftigen, lohnt sich ein genauer Blick auf modernes JavaScript. TypeScript baut vollständig auf JavaScript auf. Wer JavaScript gut versteht, wird deshalb viele TypeScript-Konzepte später unmittelbar wiedererkennen.

In diesem Kapitel schreiben wir bereits kleine Programme und probieren sie direkt in einer Angular-Anwendung aus. Dadurch können alle Beispiele unmittelbar ausgeführt und verändert werden.

Wir verwenden dabei eine **Standalone Component**, wie sie in modernen Angular-Anwendungen üblich ist.

2.1 Unser Spielplatz: eine Angular Component

Für die folgenden Beispiele verwenden wir eine Komponente `JavaScriptBasicsComponent`.

Erzeuge zunächst eine neue Komponente:

```
ng generate component components/javascript-basics
```

In einer aktuellen Angular-Anwendung entsteht dabei eine Standalone Component. Für die ersten Beispiele benötigen wir nur die TypeScript-Datei und das HTML-Template.

Eine einfache Ausgangsversion könnte so aussehen:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-javascript-basics',
  templateUrl: './javascript-basics.component.html'
})
export class JavascriptBasicsComponent {

}
```

Im Template:

```
<h1>JavaScript-Grundlagen</h1>

<p>Unsere erste Angular Component als JavaScript-Spielplatz.</p>
```

Anschließend kann die Komponente beispielsweise in einer anderen Komponente verwendet werden:

```
<app-javascript-basics />
```

Damit haben wir einen festen Ort, an dem wir die JavaScript-Beispiele dieses Kapitels ausprobieren können.

Merke: Obwohl wir mit JavaScript beginnen, befindet sich unser Code bereits in einer TypeScript-Datei. Angular verwendet TypeScript standardmäßig. Die JavaScript-Sprachelemente, die wir jetzt kennenlernen, bilden jedoch die Grundlage unseres späteren TypeScript-Codes.

2.2 Variablen mit `let` und `const`

Programme müssen Informationen speichern können. Dafür verwenden wir Variablen.

Modernes JavaScript stellt dafür vor allem zwei Schlüsselwörter bereit:

- `let`
- `const`

Ein Beispiel:

```
let name = 'Anna';  
const age = 17;
```

`let` wird verwendet, wenn sich der Wert einer Variable später verändern soll.

```
let points = 10;  
  
points = 20;
```

Bei einer mit `const` deklarierten Variable darf die Variable nicht auf einen anderen Wert zeigen:

```
const school = 'HTL';
```

Folgender Code ist deshalb nicht erlaubt:

```
school = 'Universität';
```

Beispiel in unserer Angular Component

Wir können die Variablen direkt als Eigenschaften unserer Component verwenden:

```
export class JavascriptBasicsComponent {  
  
  name = 'Anna';  
  age = 17;  
  points = 10;  
  
}
```

Im Template können wir die Werte anzeigen:

```
<h1>JavaScript-Grundlagen</h1>  
  
<p>Name: {{ name }}</p>  
<p>Alter: {{ age }}</p>  
<p>Punkte: {{ points }}</p>
```

Angular ersetzt die Ausdrücke innerhalb von `{{ }}` durch die entsprechenden Werte. Wir lernen das Konzept von *Datenbindung* (*Data-Binding*) mittels **Interpolation** später noch genauer kennen.

Das Ergebnis könnte beispielsweise sein:

```
JavaScript-Grundlagen  
  
Name: Anna  
Alter: 17  
Punkte: 10
```

Werte verändern

Wir können auch eine Methode definieren:

```
export class JavascriptBasicsComponent {  
  
  points = 10;  
  
  increasePoints() {  
    this.points = this.points + 10;  
  }  
  
}
```

Im Template:

```
<p>Punkte: {{ points }}</p>

<button (click)="increasePoints()">
  Punkte erhöhen
</button>
```

Bei jedem Klick wird die Methode ausgeführt. Dies wird durch **Event-Binding** ermöglicht. Die Schreibweise: Ein "Attribut in runden Klammern" (`click`) bekommt einen JavaScript-Funktionsaufruf zugewiesen, welcher in doppelten Hochkommas angeführt wird.

Wir haben hier bereits mehrere JavaScript-Konzepte miteinander kombiniert:

1. eine Variable,
2. eine Funktion bzw. Methode,
3. eine Zuweisung,
4. eine Berechnung.

Angular sorgt dafür, dass die Änderung anschließend im Template sichtbar wird.

2.3 Datentypen

JavaScript kennt verschiedene Datentypen.

Zu den wichtigsten gehören:

```
const name = 'Anna';
const age = 17;
const student = true;
```

Hier haben wir:

Variable	Wert	Datentyp
name	'Anna'	String
age	17	Number
student	true	Boolean

Versuche z.B. mit `typeof name` die Datentypen mittels Interpolation anzuzeigen.

JavaScript besitzt außerdem spezielle Werte wie:

```
undefined
null
```

Strings

Strings enthalten Text.

```
const firstName = 'Anna';  
const city = "Wien";
```

Beide Schreibweisen sind möglich.

Eine weitere Möglichkeit sind sogenannte **Template Literals**:

```
const name = 'Anna';  
const text = `Hallo ${name}!`;
```

Der Inhalt von `${name}` wird in den String eingesetzt.

Beispiel in Angular

```
export class JavascriptBasicsComponent {  
  
  firstName = 'Anna';  
  city = 'Wien';  
  
  get greeting() {  
    return `Hallo ${this.firstName} aus ${this.city}!`;  
  }  
  
}
```

Template:

```
<h2>{{ greeting }}</h2>
```

Das Ergebnis:

```
Hallo Anna aus Wien!
```

Template Literals werden wir später sehr häufig verwenden.

2.4 Operatoren

Mit Operatoren können wir Werte verarbeiten.

Arithmetische Operatoren

```
const a = 10;
const b = 3;

const addition = a + b;
const subtraction = a - b;
const multiplication = a * b;
const division = a / b;
const remainder = a % b;
```

Das Ergebnis von

```
10 % 3
```

ist:

```
1
```

Der `%`-Operator liefert den Rest einer Division.

Beispiel

```
export class JavascriptBasicsComponent {

  a = 10;
  b = 3;

  sum = this.a + this.b;
  difference = this.a - this.b;
  product = this.a * this.b;
  quotient = this.a / this.b;

}
```

Template:

```
<h2>Berechnungen</h2>

<p>Addition: {{ sum }}</p>
<p>Subtraktion: {{ difference }}</p>
```

```
<p>Multiplikation: {{ product }}</p>  
<p>Division: {{ quotient }}</p>
```

2.5 Vergleichsoperatoren

Vergleichsoperatoren liefern einen Boolean-Wert:

```
const age = 17;  
  
const adult = age >= 18;
```

`adult` enthält in diesem Beispiel:

```
false
```

Zu den wichtigsten Vergleichsoperatoren gehören:

```
===    gleich  
!==    ungleich  
>      größer  
<      kleiner  
>=     größer oder gleich  
<=     kleiner oder gleich
```

`===` statt `==`

In modernem JavaScript sollte grundsätzlich der strikte Vergleich verwendet werden:

```
10 === 10
```

Das Ergebnis ist:

```
true
```

Dagegen:

```
10 === '10'
```

ergibt:

```
false
```

Denn eine Zahl und ein String sind unterschiedliche Datentypen.

2.6 Bedingungen mit `if`

Mit `if` können wir Entscheidungen treffen.

```
const age = 17;

if (age >= 18) {
  console.log('Volljährig');
} else {
  console.log('Minderjährig');
}
```

Die Bedingung

```
age >= 18
```

wird ausgewertet.

Ist sie `true`, wird der erste Block ausgeführt. Andernfalls der `else`-Block.

Beispiel in der Component

```
export class JavascriptBasicsComponent {  
  
  age = 17;  
  
  get ageMessage() {  
  
    if (this.age >= 18) {  
      return 'Volljährig';  
    } else {  
      return 'Minderjährig';  
    }  
  
  }  
  
}
```

Template:

```
<p>Alter: {{ age }}</p>  
<p>{{ ageMessage }}</p>
```

Wir können den Wert anschließend verändern:

```
age = 21;
```

Das Ergebnis ändert sich automatisch.

2.7 Mehrere Bedingungen

Mit `else if` können mehrere Fälle unterschieden werden.

```
const points = 82;  
  
if (points >= 90) {  
  console.log('Sehr gut');  
} else if (points >= 76) {  
  console.log('Gut');  
} else if (points >= 62) {  
  console.log('Befriedigend');  
} else if (points >= 51) {  
  console.log('Genügend');  
} else {
```

```
console.log('Nicht genügend');  
}
```

Eine solche Struktur können wir auch direkt in einer Component verwenden.

```
export class JavascriptBasicsComponent {  
  
  points = 82;  
  
  get grade() {  
  
    if (this.points >= 90) {  
      return 'Sehr gut';  
    }  
  
    if (this.points >= 76) {  
      return 'Gut';  
    }  
  
    if (this.points >= 62) {  
      return 'Befriedigend';  
    }  
  
    if (this.points >= 51) {  
      return 'Genügend';  
    }  
  
    return 'Nicht genügend';  
  }  
  
}
```

Template:

```
<p>Punkte: {{ points }}</p>  
<p>Bewertung: {{ grade }}</p>
```

2.8 Logische Operatoren

Bedingungen können miteinander kombiniert werden.

Der logische UND-Operator lautet:

```
&&
```

Beispiel:

```
const age = 20;  
const hasTicket = true;  
  
const canEnter = age >= 18 && hasTicket;
```

Beide Bedingungen müssen **true** sein.

Der logische ODER-Operator lautet:

```
||
```

Beispiel:

```
const weekend = true;  
const holiday = false;  
  
const dayOff = weekend || holiday;
```

Mindestens eine Bedingung muss **true** sein.

Mit **!** kann ein Boolean umgekehrt werden:

```
const locked = false;  
const unlocked = !locked;
```

2.9 Funktionen

Funktionen kapseln Anweisungen, die wir mehrfach verwenden können.

Eine einfache Funktion:

```
function greet() {  
  return 'Hallo!';  
}
```

Eine Funktion kann Parameter besitzen:

```
function greet(name) {  
  return `Hallo ${name}!`;  
}
```

Aufruf:

```
greet('Anna');
```

Ergebnis:

Hallo Anna!

Funktionen in unserer Component

```
export class JavascriptBasicsComponent {  
  
  greet(name: string) {  
    return `Hallo ${name}!`;  
  }  
  
}
```

Template:

```
<p>{{ greet('Anna') }}</p>  
<p>{{ greet('Max') }}</p>
```

Hier sehen wir bereits einen kleinen Unterschied zwischen JavaScript und TypeScript:

```
name: string
```

Die Typangabe werden wir im nächsten Kapitel ausführlich kennenlernen.

2.10 Arrow Functions

Moderne JavaScript-Anwendungen verwenden sehr häufig sogenannte Arrow Functions.

Die klassische Schreibweise:

```
function add(a, b) {  
  return a + b;  
}
```

kann als Arrow Function geschrieben werden:

```
const add = (a, b) => {  
  return a + b;  
};
```

Wenn nur ein Ausdruck zurückgegeben wird, kann die Funktion noch kürzer geschrieben werden:

```
const add = (a, b) => a + b;
```

Das ist besonders praktisch bei Arrays.

2.11 Arrays

Ein Array enthält mehrere Werte.

```
const names = ['Anna', 'Max', 'Lisa'];
```

Auf einzelne Elemente kann über den Index zugegriffen werden:

```
names[0]
```

liefert:

```
Anna
```

Wichtig: Die Zählung beginnt bei 0.

```
Index 0 → Anna  
Index 1 → Max  
Index 2 → Lisa
```

Array in einer Angular Component

```
export class JavascriptBasicsComponent {  
  
  names = ['Anna', 'Max', 'Lisa'];  
  
}
```

Im Template können wir zunächst einzelne Elemente anzeigen:

```
<p>{{ names[0] }}</p>  
<p>{{ names[1] }}</p>  
<p>{{ names[2] }}</p>
```

2.12 Array-Methoden

Arrays besitzen viele nützliche Methoden.

Eine der einfachsten ist `push()`:

```
const names = ['Anna', 'Max'];  
  
names.push('Lisa');
```

Das Array enthält anschließend:

```
Anna  
Max  
Lisa
```

Die Anzahl der Elemente erhalten wir mit:

```
names.length
```

Beispiel

```
export class JavascriptBasicsComponent {  
  
  names = ['Anna', 'Max'];
```

```
addName() {  
  this.names.push('Lisa');  
}  
  
}
```

Template:

```
<p>Anzahl: {{ names.length }}</p>  
  
<button (click)="addName()">  
  Lisa hinzufügen  
</button>  
  
<p>{{ names.join(', ') }}</p>
```

`join()` verbindet die Elemente eines Arrays zu einem String.

2.13 `map()`

Eine besonders wichtige Array-Methode ist `map()`.

Angenommen, wir haben:

```
const numbers = [1, 2, 3, 4];
```

Wir möchten jede Zahl verdoppeln.

```
const doubled = numbers.map(number => number * 2);
```

Das Ergebnis:

```
[2, 4, 6, 8]
```

`map()` erzeugt dabei ein neues Array.

In unserer Component

```
export class JavascriptBasicsComponent {
```

```
numbers = [1, 2, 3, 4];

doubledNumbers = this.numbers.map(
  number => number * 2
);

}
```

Template:

```
<p>Original: {{ numbers.join(', ') }}</p>

<p>Verdoppelt: {{ doubledNumbers.join(', ') }}</p>
```

2.14 filter()

Mit `filter()` können wir Elemente auswählen, die eine bestimmte Bedingung erfüllen.

```
const numbers = [1, 2, 3, 4, 5, 6];

const evenNumbers = numbers.filter(
  number => number % 2 === 0
);
```

Das Ergebnis:

```
[2, 4, 6]
```

Angular-Beispiel

```
export class JavascriptBasicsComponent {

  numbers = [1, 2, 3, 4, 5, 6];

  evenNumbers = this.numbers.filter(
    number => number % 2 === 0
  );

}
```

Template:

```
<p>Zahlen: {{ numbers.join(', ') }}</p>

<p>Gerade Zahlen: {{ evenNumbers.join(', ') }}</p>
```

2.15 find()

Mit `find()` suchen wir das erste Element, das eine Bedingung erfüllt.

```
const numbers = [5, 12, 8, 20];

const result = numbers.find(
  number => number > 10
);
```

Das Ergebnis ist:

```
12
```

Die Methode `find()` beendet die Suche, sobald das erste passende Element gefunden wurde.

2.16 Objekte

Ein Objekt fasst mehrere zusammengehörige Informationen zusammen.

```
const student = {
  name: 'Anna',
  age: 17,
  class: '5AHIT'
};
```

Auf Eigenschaften können wir über den Punktoperator zugreifen:

```
student.name
student.age
student.class
```

Objekt in einer Component

```
export class JavascriptBasicsComponent {  
  
  student = {  
    name: 'Anna',  
    age: 17,  
    class: '5AHIT'  
  };  
  
}
```

Template:

```
<h2>Schüler</h2>  
  
<p>Name: {{ student.name }}</p>  
<p>Alter: {{ student.age }}</p>  
<p>Klasse: {{ student.class }}</p>
```

Hier wird bereits deutlich, warum strukturierte Daten für Anwendungen wichtig sind.

2.17 Arrays mit Objekten

In echten Anwendungen speichern wir häufig mehrere Objekte in einem Array.

```
export class JavascriptBasicsComponent {  
  
  students = [  
    {  
      name: 'Anna',  
      age: 17  
    },  
    {  
      name: 'Max',  
      age: 18  
    },  
    {  
      name: 'Lisa',  
      age: 16  
    }  
  ];  
  
}
```

Damit haben wir eine kleine Datenstruktur erstellt:

```
students
├── Student
│   ├── name
│   └── age
├── Student
│   ├── name
│   └── age
└── Student
    ├── name
    └── age
```

In Angular möchten wir solche Listen natürlich nicht einzeln ausgeben.

Dafür lernen wir später die moderne `@for`-Syntax kennen.

Zum jetzigen Zeitpunkt können wir beispielsweise zunächst einen einzelnen Datensatz anzeigen:

```
<p>{{ students[0].name }}</p>
```

2.18 Destructuring

JavaScript bietet eine elegante Möglichkeit, Werte aus Objekten herauszuholen.

Angenommen:

```
const student = {
  name: 'Anna',
  age: 17
};
```

Ohne Destructuring schreiben wir:

```
const name = student.name;
const age = student.age;
```

Mit Destructuring geht es kürzer:

```
const { name, age } = student;
```

Danach stehen `name` und `age` direkt zur Verfügung.

Beispiel

```
export class JavascriptBasicsComponent {  
  
  student = {  
    name: 'Anna',  
    age: 17  
  };  
  
  getStudentDescription() {  
  
    const { name, age } = this.student;  
  
    return `${name} ist ${age} Jahre alt.`;  
  }  
  
}
```

Template:

```
<p>{{ getStudentDescription() }}</p>
```

2.19 Spread Operator

Mit dem Spread Operator `...` können Inhalte eines Arrays oder Objekts übernommen werden.

Beispiel:

```
const numbers = [1, 2, 3];  
  
const moreNumbers = [...numbers, 4, 5];
```

Das Ergebnis:

```
[1, 2, 3, 4, 5]
```

Der ursprüngliche Inhalt wird dabei nicht verändert.

Auch bei Objekten ist der Spread Operator nützlich:

```
const student = {  
  name: 'Anna',
```

```
    age: 17
  };

  const olderStudent = {
    ...student,
    age: 18
  };

```

Das neue Objekt enthält:

```
name: Anna
age: 18

```

2.20 Nullish Coalescing

Manchmal kann ein Wert `null` oder `undefined` sein.

JavaScript bietet dafür den Operator `??`.

```
const name = null;

const displayName = name ?? 'Unbekannt';

```

Das Ergebnis lautet:

```
Unbekannt

```

Der rechte Wert wird verwendet, wenn der linke Wert `null` oder `undefined` ist.

Das ist beispielsweise bei Daten aus einer API sehr praktisch.

2.21 Optional Chaining

Mit dem Operator `?.` können wir sicher auf verschachtelte Eigenschaften zugreifen.

Angenommen:

```
const student = null;

```

Ein direkter Zugriff wäre problematisch:

```
student.name
```

Mit Optional Chaining:

```
student?.name
```

erhalten wir stattdessen `undefined`.

Bei verschachtelten Objekten:

```
student?.address?.city
```

wird nicht mit einem Fehler abgebrochen, wenn `student` oder `address` nicht vorhanden ist.

2.22 Klassen

JavaScript unterstützt objektorientierte Programmierung.

Eine Klasse:

```
class Person {  
  
  name;  
  
  constructor(name) {  
    this.name = name;  
  }  
  
  greet() {  
    return `Hallo ${this.name}!`;  
  }  
  
}
```

Wir können daraus ein Objekt erzeugen:

```
const person = new Person('Anna');  
  
person.greet();
```

Klassen werden wir in TypeScript wiederfinden. Dort bietet die Sprache zusätzliche Möglichkeiten für eine sichere Typisierung.

In Angular sind Komponenten selbst Klassen:

```
export class JavascriptBasicsComponent {  
  
}
```

Die Klasse beschreibt das Verhalten und den Zustand unserer Komponente.

2.23 Module

Moderne JavaScript-Anwendungen bestehen nicht aus einer einzigen großen Datei.

Code wird auf mehrere Module verteilt.

Ein Modul kann beispielsweise eine Funktion exportieren:

```
export function add(a, b) {  
  return a + b;  
}
```

Ein anderes Modul kann sie importieren:

```
import { add } from './math';
```

Angular-Anwendungen bestehen ebenfalls aus vielen Modulen bzw. Dateien, die zusammen eine Anwendung bilden.

Bei modernen Angular-Anwendungen spielt dabei die Standalone-Architektur eine wichtige Rolle.

2.24 Promises

Viele Aufgaben in Webanwendungen laufen nicht sofort ab.

Ein typisches Beispiel ist das Laden von Daten von einem Server.

JavaScript verwendet dafür unter anderem Promises.

Ein einfaches Promise:

```
const promise = new Promise(resolve => {  
  setTimeout(() => {  
    resolve('Daten geladen');  
  }, 1000);  
});
```

Ein Promise kann später erfolgreich abgeschlossen werden.

Mit `then()` können wir auf das Ergebnis reagieren:

```
promise.then(result => {  
  console.log(result);  
});
```

Nach ungefähr einer Sekunde wird ausgegeben:

```
Daten geladen
```

2.25 `async` und `await`

Eine moderne und gut lesbare Schreibweise für asynchronen Code ist `async/await`.

```
async function loadData() {  
  const result = await promise;  
  console.log(result);  
}
```

`await` wartet auf das Ergebnis eines Promise.

In Angular werden wir später asynchrone Daten häufig über HTTP und RxJS verarbeiten. Die Grundlagen von Promises bleiben trotzdem wichtig, weil viele Web-APIs darauf aufbauen.

2.26 `setTimeout()` als erstes asynchrones Beispiel

Zum Experimentieren können wir `setTimeout()` verwenden.

```
export class JavascriptBasicsComponent {  
  
  message = 'Bitte warten ...';  
  
  loadMessage() {  
  
    setTimeout(() => {  
      this.message = 'Daten sind angekommen!';  
    }, 2000);  
  
  }  
  
}
```

Template:

```
<p>{{ message }}</p>  
  
<button (click)="loadMessage()">  
  Daten laden  
</button>
```

Beim Klick erscheint zunächst:

Bitte warten ...

Nach zwei Sekunden:

Daten sind angekommen!

Damit haben wir bereits einen wichtigen Aspekt moderner Webanwendungen kennengelernt: **Code kann zeitversetzt ausgeführt werden.**

2.27 Ein kleines Abschlussprojekt

Zum Abschluss verbinden wir mehrere Konzepte dieses Kapitels.

Wir erstellen eine kleine Punkteliste.

Component

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-javascript-basics',
  templateUrl: './javascript-basics.component.html'
})
export class JavascriptBasicsComponent {

  studentName = 'Anna';

  points = [75, 90, 62, 88];

  addPoints() {
    this.points = [...this.points, 95];
  }

  get average() {

    const sum = this.points.reduce(
      (total, point) => total + point,
      0
    );

    return sum / this.points.length;
  }

  get result() {

    if (this.average >= 90) {
      return 'Sehr gut';
    }

    if (this.average >= 76) {
      return 'Gut';
    }

    if (this.average >= 62) {
      return 'Befriedigend';
    }

    if (this.average >= 51) {
      return 'Bestanden';
    }

    return 'Nicht bestanden';
  }
}
```

Das Template:

```
<h1>Leistungsübersicht</h1>

<h2>{{ studentName }}</h2>

<p>
  Punkte:
  {{ points.join(', ') }}
</p>

<p>
  Durchschnitt:
  {{ average.toFixed(2) }}
</p>

<p>
  Ergebnis:
  {{ result }}
</p>

<button (click)="addPoints()">
  Weitere Prüfung hinzufügen
</button>
```

Hier verwenden wir bereits viele JavaScript-Konzepte:

- Variablen
- Arrays
- Methoden
- Arrow Functions
- `reduce()`
- Template Literals bzw. Strings
- Bedingungen
- Spread Operator
- Berechnungen
- Objekte bzw. Klassenstruktur
- asynchrone Konzepte haben wir vorbereitet

2.28 Übungen

Übung 1 – Persönliche Daten

Erweitere die Component um folgende Informationen:

```
Vorname
Nachname
```

```
Alter  
Wohnort
```

Zeige alle Informationen im Template an.

Übung 2 – Taschenrechner

Erstelle zwei Variablen:

```
number1  
number2
```

Implementiere Methoden für:

- Addition
- Subtraktion
- Multiplikation
- Division

Zeige die Ergebnisse im Template an.

Übung 3 – Altersprüfung

Erstelle eine Variable:

```
age = 16;
```

Gib abhängig vom Alter eine Meldung aus:

```
unter 14 → Kind  
14 bis 17 → Jugendliche/r  
ab 18 → Erwachsene/r
```

Übung 4 – Zahlen filtern

Erstelle das Array:

```
numbers = [4, 7, 12, 19, 20, 25, 30];
```

Erzeuge mit `filter()` ein neues Array, das nur die geraden Zahlen enthält.

Übung 5 – Preise

Erstelle ein Array mit Preisen:

```
prices = [10, 25, 7.5, 100];
```

Verwende `map()`, um alle Preise um 20 % zu erhöhen.

Übung 6 – Schülerinnen und Schüler

Erstelle ein Array aus Objekten:

```
students = [  
  { name: 'Anna', age: 17 },  
  { name: 'Max', age: 18 },  
  { name: 'Lisa', age: 16 },  
  { name: 'Paul', age: 19 }  
];
```

Verwende `filter()`, um nur die volljährigen Personen zu erhalten.

Übung 7 – Durchschnitt

Erstelle ein Array mit fünf Prüfungsergebnissen.

Berechne mit `reduce()` den Durchschnitt.

Zeige anschließend an:

```
Durchschnitt: 78.40
```

Übung 8 – Eigene Idee

Erstelle eine kleine Component, die ein Problem aus deinem Alltag löst.

Beispiele:

- Taschengeldrechner
- Notenrechner
- Temperaturumrechner
- Einkaufslisten-Rechner
- BMI-Rechner

- Fahrkostenrechner

Verwende dabei mindestens:

- eine Variable,
- eine Methode,
- eine Bedingung,
- ein Array,
- eine Array-Methode.

2.29 Zusammenfassung

In diesem Kapitel haben wir die wichtigsten Grundlagen von modernem JavaScript kennengelernt.

Besonders wichtig sind:

```
let / const
Datentypen
Operatoren
if / else
Funktionen
Arrow Functions
Arrays
Objekte
map()
filter()
find()
reduce()
Destructuring
Spread Operator
Optional Chaining
Nullish Coalescing
Klassen
Module
Promises
async / await
```

Die Beispiele wurden bewusst innerhalb einer Angular Component umgesetzt. Dadurch sehen wir bereits, wie JavaScript-Code in einer modernen Webanwendung eingesetzt wird.

Dabei ist eine wichtige Beobachtung entstanden:

Angular ersetzt JavaScript nicht. Angular baut auf JavaScript auf.

Angular stellt zusätzliche Konzepte für die Entwicklung großer Webanwendungen bereit. Die eigentliche Programmiersprache bleibt JavaScript – beziehungsweise TypeScript, das JavaScript um ein leistungsfähiges Typsystem und weitere Sprachfeatures erweitert.

Im nächsten Kapitel machen wir deshalb den entscheidenden Schritt:

Aus JavaScript wird TypeScript.

Wir werden sehen, wie TypeScript Variablen, Funktionen, Arrays und Objekte typisiert und warum diese zusätzliche Information gerade bei größeren Angular-Anwendungen so wertvoll ist.